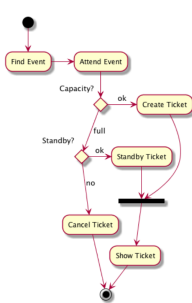


Chap 1 Intro ...

Qualité d'un logiciel

- Maintenabilité
- Réparabilité
- Evolutivité
- Réutilisation
- Portabilité
- Interopérabilité (Capacité à être intégré dans un autre system)

```
(* ) --> "Find Event"  
"Find Event" -> "Attend Event"  
  
if "Capacity?" then  
  -->[ok] "Create Ticket"  
else  
  -->[full] if "Standby?" then  
    -->[ok] "Standby Ticket"  
  else  
    -->[no] "Cancel Ticket"  
    "Cancel Ticket" --> (*)  
  endif  
endif  
endif  
  
"Create Ticket" --> ==show==  
"Standby Ticket" --> ==show==  
==show== --> "Show Ticket"  
"Show Ticket" --> (*)
```



System Requirement

Just un doc avec les requirement :

- Bisness : en gros ce que veut l'investisseur parceque \$
- utilisateur
- system : technique

Functionnels vs non-fonctionnels

Fonctionnels : en gros, c'est le comment, genre comment le system doit faire X

Non-fonctionnels : C'est les contraintes des fonctions du system, genre quel techno utiliser, standard à utiliser ou la vitesse d'exécution (c'est mieux si c'est quantitatif pour tester facilement)

Élicitation → Spécification → Validation

En gros, l'idée c'est de split les valeurs ajoutées du projet, exemple avec "une app de note" (élément après interview) :

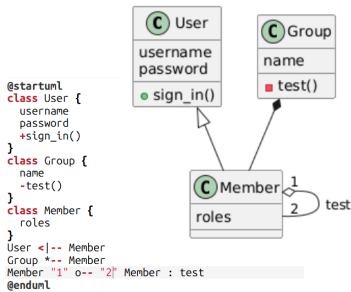
- Pouvoir prendre des notes | cas d'utilisation
- Éviter de perdre des notes | contrainte (parce qu'il n'y a pas de valeur ajoutée)
- pouvoir partager ses notes avec d'autres appareils | cas d'utilisation
- ce login | étape

Histoire

Just un doc avec le narratif derrière le projet

Scénario

- état initial
- flux d'événements
- cas d'échecs
- activités concurrentes
- état final



Peut se faire avec un UML ou un tableau, en vrai ça dépend de la cible

Chap 4 Modélisation du système

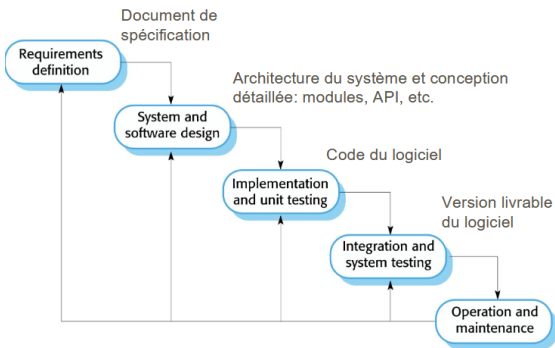
- Modèle de contexte et de processus : Environnement du system
- Modèle d'interactions : Interaction entre enviro et le system (ou composant)
- Modèle structurels : Organisation et structure du system
- Modèle de comportement : Comportement du system

Modèle de Processus logiciel

Modèle Cascade (planifiés)

On prévoit tous les tickets

Phase



Modèle Incrémentaux (agiles)

On prévoit pas trop et change au fur et à mesure pour s'adapter

Modèle Intégration et de configuration

En gros, on essaye de gagner du temps en réutilisant ce qu'on a fait avant

Chap 3 Requirements Engineering

1. évaluer les services que le client demande du system
2. Contraintes sous lesquels il opère et est développé

Diagramme de use case : Interaction system et enviro

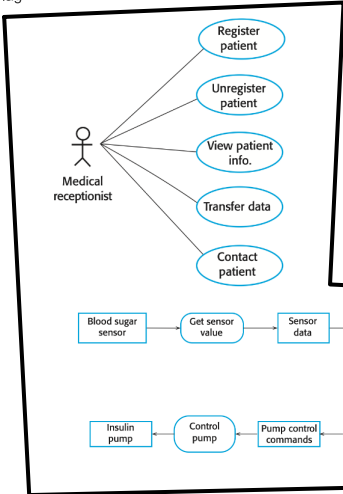
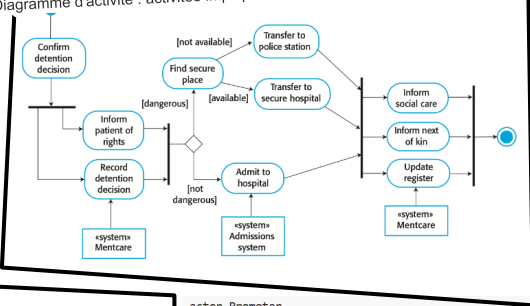


Diagramme d'activité : activités impliquées dans le processus ou le traitement de données



actor Promoter

actor Entrant

Promoter --> (Create Event)

Promoter --> (Attend Event)

Entrant --> (Find Event)

(Attend Event) <- Entrant

(Attend Event) <.. (Create Member) : <<include>>

Diagramme de séquence : interaction entre acteurs et system

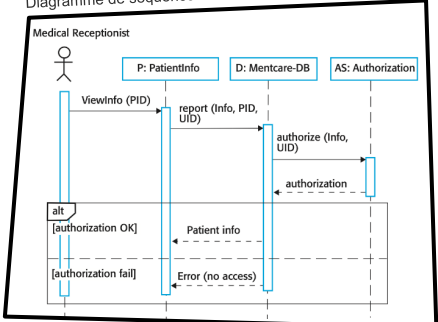
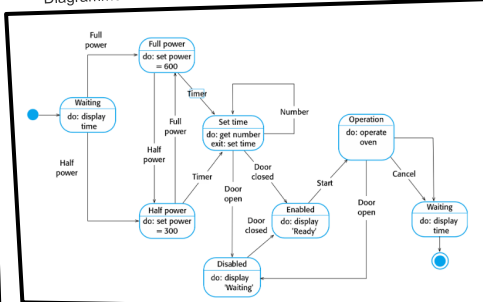
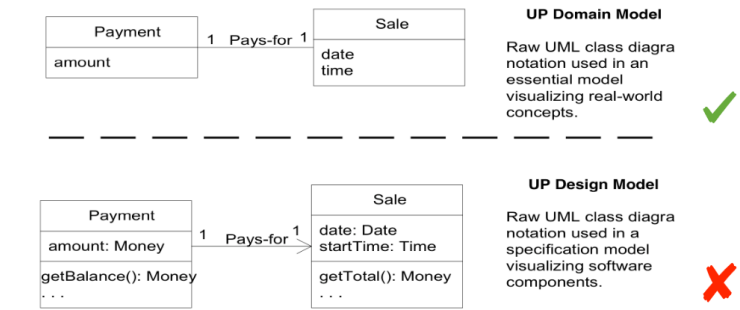


Diagramme d'état : comment le system réagit aux événements

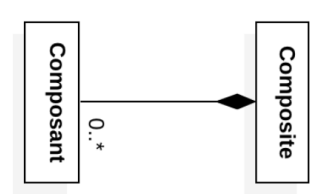


Chap 5,6,7,8

Modélisation du domaine

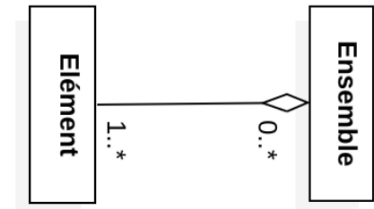


Composition



Pas de partage, exemple :
moteur (composant) → voiture (composite)

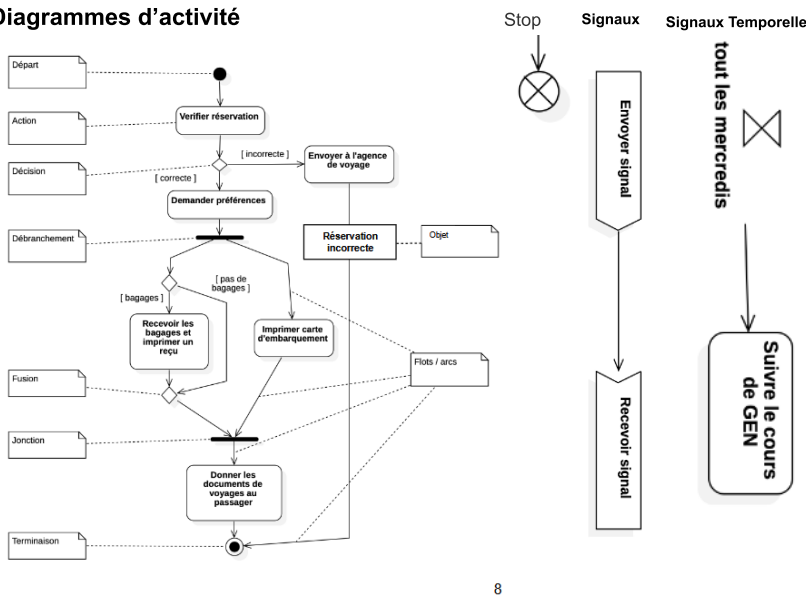
Aggregation



Partage, exemple :
Voiture (élément) → Honda (ensemble)

A fait partie de B, genre si A explose, B aussi :[

Diagrammes d'activité

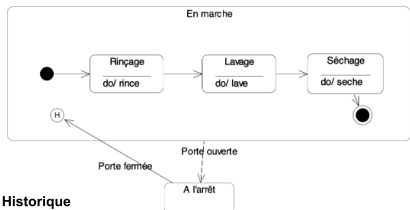


Modèles de comportement (séance)

IF

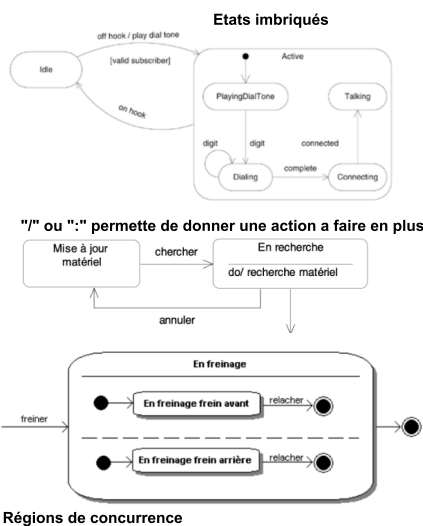


Etats imbriqués



Historique

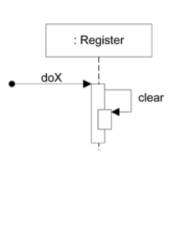
Le H revient a l'état d'où on est sortie
H* = revient a l'état de sortie des etats imbriqués



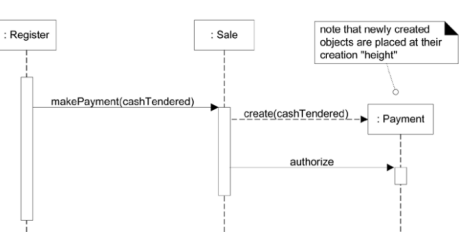
Régions de concurrence

Modèles de comportement (séance)

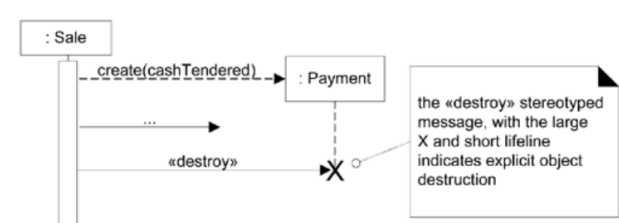
exécution imbriquée



Création d'une instance

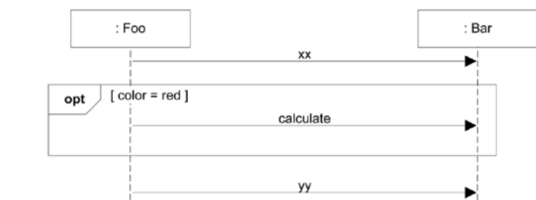


Destruction d'une instance

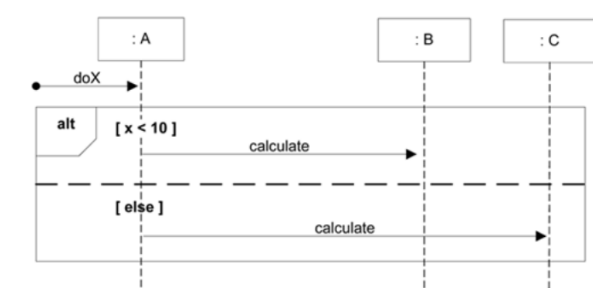


"Operator"

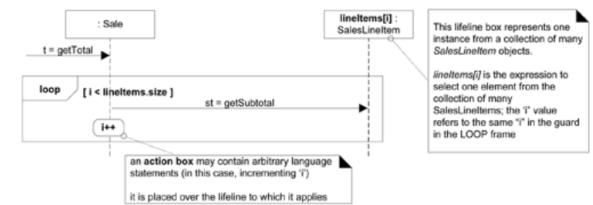
- opt = if



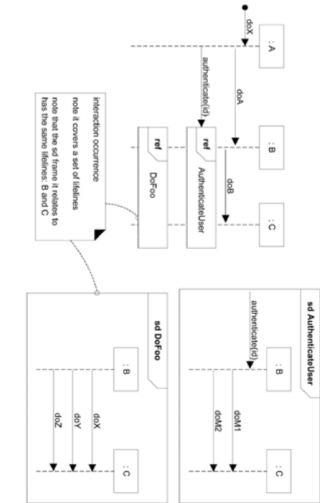
- alt = if else



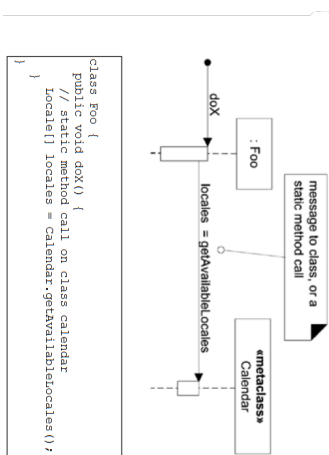
- loop = for(n)



Sous-diagramme



Methods static



• par = thread/parallèle | region = region critique dans les thread | ref "name" = include "name" (utiliser pour les sous diagramme)